Smart Museum - Person 3 Frontend PWA Technical Handoff

Project: Smart Museum - Cultural Heritage Monitoring System
Document: Person 3 - Frontend PWA Technical Handoff
Role: P3 - Client Application / Frontend
Technology: React 19 and Vite 7
Version: 1.0
Prepared: 2 November 2026
Audience: Person 1 (IoT/Node-RED), Person 2 (Jakarta EE backend), project team, and professor

> **Verification status:** This document describes the source currently in `pwa-client/`. Automated tests and the production build were rerun on 2 November 2026. The backend source and `contracts/openapi.yaml` are not in this repository, so live API compatibility and CORS are not verified.

Contents

1. Project context
2. P3 scope and completed deliverables
3. Technical architecture
4. Repository structure and important files
5. Installation and local execution
6. API integration contract and backend expectations
7. Mock data and demonstration behavior
8. Data freshness and refresh behavior
9. PWA implementation
10. Testing and quality assurance
11. UML and technical diagrams
12. Known limitations and pending decisions
13. Integration procedure after P2 publishes the backend
14. Handoff summary and responsibilities

1. Project context

Smart Museum is a cultural-heritage monitoring project intended to give museum personnel environmental context for exhibition areas and collections. Temperature, humidity, pressure, and illuminance are environmental properties that a conservation team may review when caring for artifacts. The PWA displays measurements only when records are available; it does not assess conservation compliance or make preservation recommendations.

The client-facing Progressive Web App (PWA) provides a responsive browser interface for staff, curators, and conservation personnel. It can run in demonstration mode without the backend. Its API mode is prepared for the documented read endpoints but has not been tested against a live backend.

Team boundaries

Owner	Project responsibility	P3 boundary
P1	Sensors, gateway/edge environment, Node-RED and IoT-side telemetry flow	P3 consumes read results; it does not configure sensors, MQTT, gateways, or ingestion.
P2	Jakarta EE backend, persistence, and shared API contract	P3 calls the documented read endpoints only. P3 does not implement the backend, database, or API contract.
P3	React client PWA and frontend/conceptual documentation	Dashboard, mock-mode development, API client boundary, client-side presentation, and PWA shell.

Measurement **ingestion** and PWA **retrieval** are separate flows. The conceptual ingestion path is sensor -> gateway/Node-RED -> backend -> MongoDB. The PWA read path is browser/PWA -> backend REST API -> MongoDB read handled by the backend -> API response -> browser/PWA. P3 owns only the browser client.

2. P3 scope and completed deliverables

The following implementation is present in the inspected source. "Implemented" describes frontend behavior and does not imply a verified live backend.

Feature	Behavior in the client	Main implementation
Dashboard and navigation	Smart Museum sidebar, dashboard heading, responsive content, section anchors, connection indicators	`src/pages/ConservationDashboard.jsx`, `src/components/Sidebar.jsx`, `src/styles/main.css`
Zone selection	Selects `salle-01` or `salle-02`; the latest-data effect reruns for the selected zone	`ConservationDashboard.jsx`, `src/services/dataSource.js`
Measurement cards	Renders property groups with value, unit, `observedAt`, device, zone, source, optional gateway/quality, and freshness where fields exist	`src/components/MeasurementCard.jsx`, `src/utils/measurementData.js`
Environmental properties	Supports `temperature`, `humidity`, `pressure`, and `illuminance`; only returned properties are rendered	`measurementData.js`, `ConservationDashboard.jsx`
Device and zone information	Shows available device ID, zone ID, gateway ID, and source. No online/status field is invented.	`src/components/ZoneDetails.jsx`
Data provenance	Separates `real` and `simulated` source labels from the prominent mock/demo label. A mock record with `source: real` remains demonstration data.	`src/components/StatusMessage.jsx`, `MeasurementCard.jsx`, `mockData.js`
Freshness	Computes freshness from measurement `observedAt`; invalid, future, or absent timestamps do not appear current	`src/utils/measurements.js`, `FreshnessIndicator.jsx`, `ConservationDashboard.jsx`

Feature	Behavior in the client	Main implementation
Automatic refresh	Starts an immediate read and repeats at a configurable interval; prevents overlapping runs and cleans up on unmount	`src/services/refreshScheduler.js`, `ConservationDashboard.jsx`
Historical measurements	Zone/device/property/date-time/limit filters; SVG chart plus accessible HTML table; result labels remain attached to the submitted query	`src/components/HistoricalMeasurements.jsx`, `HistoryChart.jsx`, `src/utils/historyChart.js`
Alerts	Placeholder only. It does not retrieve or generate alerts.	`src/components/AlertsPlaceholder.jsx`
Error and offline states	Separates API connection state, browser online/offline state, loading/empty/error states, and measurement freshness	`StatusMessage.jsx`, `PwaControls.jsx`, `HistoricalMeasurements.jsx`
PWA shell	Manifest and icons, production service-worker registration, app-shell cache, offline navigation response, install prompt and user-controlled update	`src/main.jsx`, `public/manifest.webmanifest`, `public/service-worker.js`, `public/offline.html`, `PwaControls.jsx`
Documentation and diagrams	P3 guide and editable Mermaid use-case, retrieval/ingestion sequence, and communication sources	`DOCS/pwa-client.md`, `DOCS/diagrams/*.mmd`

Explicitly not delivered

There is no backend implementation, sensor/gateway integration, ingestion client, alert API, authentication flow, role management, analytics, prediction, or anomaly detection in the P3 frontend. No service token or ingestion credential belongs in browser code.

3. Technical architecture

Frontend structure and data flow

The app is a JavaScript React 19 single-page application built and served by Vite 7. `src/main.jsx` mounts `App`; `App` renders the conservation dashboard. The dashboard selects a zone and data mode, requests data through `src/services/dataSource.js`, and composes reusable components. UI components do not make their own HTTP requests.

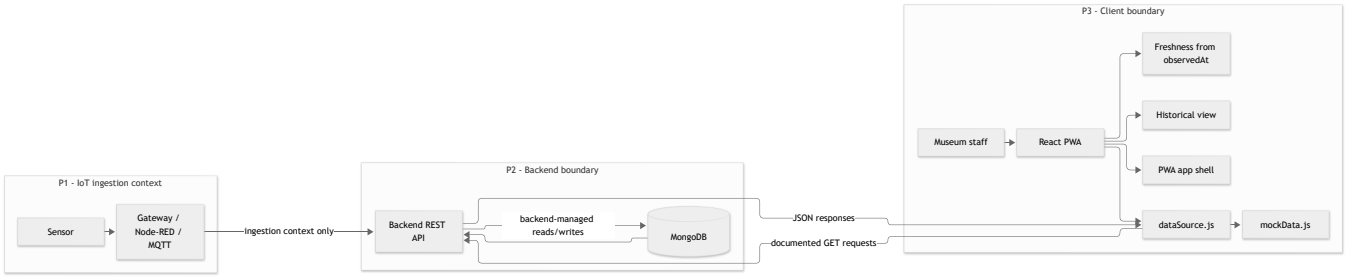


Figure 1. P3 frontend architecture and system ownership boundaries.

Responsibilities of the frontend modules

1. `ConservationDashboard.jsx` holds the selected zone and mode, schedules latest-data retrieval, exposes request/connection states, groups measurements, calculates freshness, and arranges dashboard sections.
2. `dataSource.js` selects mock or API behavior. The API path retrieves the device list and latest measurement array in parallel. The history path delegates to the central API client or filters existing demo fixtures.
3. `api.js` centralizes base URL, GET requests, timeouts, JSON parsing, response validation, cancellation, query encoding, and API error types.
4. `measurementData.js`, `measurements.js`, and `historyChart.js` prepare property metadata, source timestamps/freshness, and chart points without mutating the API records.
5. Components render the resulting state. `HistoricalMeasurements.jsx` sends a history query on explicit form submission and renders an SVG chart and a table.

System boundary

The PWA does not communicate directly with MongoDB or IoT hardware. In the documented system context, P1 supplies telemetry through its IoT/gateway path. P2 owns API persistence and read behavior. The browser issues only the documented GET operations. The architecture diagram's ingestion arrows are context, not P3 code.

4. Repository structure and important files

The following tree excludes `node_modules/` and generated `dist/` output.

```
pwa-client/
  .env.example
  index.html
  package.json
  package-lock.json
  vite.config.js
  vitest.config.js
  public/
    manifest.webmanifest
    offline.html
    service-worker.js
    icons/
      museum-mark.svg
      museum-mark-192.png
      museum-mark-512.png
  src/
    App.jsx
    main.jsx
    components/
      AlertsPlaceholder.jsx
      FreshnessIndicator.jsx
      HistoricalMeasurements.jsx
      HistoryChart.jsx
      MeasurementCard.jsx
      PwaControls.jsx
      Sidebar.jsx
      StatusMessage.jsx
      ZoneDetails.jsx
      history-dashboard.component.test.jsx
    pages/
      ConservationDashboard.jsx
    services/
      api.js
      api.test.js
      dataSource.js
      dataSource.test.js
      mockData.js
      pwa.test.js
      refreshScheduler.js
      refreshScheduler.test.js
    styles/
      main.css
    utils/
      historyChart.js
      historyChart.test.js
      measurementData.js
      measurementData.test.js
      measurements.js
      measurements.test.js

DOCS/
  pwa-client.md
  P3_Technical_Handoff_Smart_Museum.md
  diagrams/
    pwa-use-cases.mmd
    pwa-measurement-sequence.mmd
    pwa-communication.mmd
    Smart_Museum_P3_Diagrams.pdf
```

Important file responsibilities

File(s)	Responsibility and dependencies
`pwa-client/package.json`, `package-lock.json`	Scripts and locked React/Vite/Vitest/Testing Library dependencies; `npm ci` reproduces the lockfile.
`pwa-client/.env.example`	Non-secret local defaults for data mode, API URL, timeout, freshness, and refresh.
`pwa-client/vite.config.js`, `vitest.config.js`	Vite React build configuration and Vitest jsdom component test configuration.
`pwa-client/index.html`	Browser document metadata, app icons, manifest link, and React root element.
`src/main.jsx`, `src/App.jsx`	Mounts React and renders `ConservationDashboard`; production-only service-worker registration and update lifecycle are in `main.jsx`.
`src/pages/ConservationDashboard.jsx`	Main composition, zone/mode state, refresh effect, freshness summary, and connection status.
`src/components/Sidebar.jsx`, `StatusMessage.jsx`, `ZoneDetails.jsx`, `AlertsPlaceholder.jsx`	Navigation; loading/demo/error messaging; device details; alert placeholder.
`src/components/MeasurementCard.jsx`, `FreshnessIndicator.jsx`	Measurement values/metadata and aggregate freshness display.
`src/components/HistoricalMeasurements.jsx`, `HistoryChart.jsx`	History form/query state, responsive chart, and accessible data table.
`src/components/PwaControls.jsx`	Browser online/offline listeners, install prompt, and update action.
`src/services/api.js`	Central REST client for health, devices, latest measurements, and history.
`src/services/dataSource.js`, `mockData.js`	Data-mode boundary and the two documented demonstration devices/fixture records.
`src/services/refreshScheduler.js`	Immediate/interval scheduler with in-flight overlap prevention.
`src/utils/measurements.js`	Freshness threshold, refresh interval, freshness states, and timestamp formatting.
`src/utils/measurementData.js`, `historyChart.js`	Property grouping/metadata and SVG plotting coordinates.
`src/styles/main.css`	Visual system, responsive layout, and narrow-screen presentation.
`public/manifest.webmanifest`, `public/icons/*`	Install metadata and launcher icon assets.
`public/service-worker.js`, `public/offline.html`	Static app-shell caching and navigation fallback. API responses are excluded.
`src/services/*.test.js`, `src/utils/*.test.js`	Node built-in tests for services, utility behavior, and PWA assets.
`src/components/history-dashboard.component.test.jsx`	Vitest/Testing Library tests of history labels, zone switching, and data-mode controls.
`DOCS/pwa-client.md`	User-facing P3 architecture, configuration, API boundary, and operating guide.
`DOCS/diagrams/*.mmd`	Editable use-case, sequence, and communication diagrams.
`DOCS/P3_Technical_Handoff_Smart_Museum.md`	Editable source for this handoff.

5. Installation and local execution

Prerequisites

- Node.js 20.19+ or 22.12+ as stated in `pwa-client/README.md`.
- npm.
- A modern browser. Production service-worker tests require a secure context: HTTPS or localhost.

Commands (PowerShell)

```
cd "C:\Users\souro\Desktop\MBK project\pwa-client"
npm ci
Copy-Item .env.example .env
npm run dev
```

Use the local address printed by Vite. Its default development origin is `http://localhost:5173`; Vite can select another port if that port is in use. The actual printed origin is the one that must be allowed by backend CORS.

Run checks and production preview:

```
npm test
npm run build
npm run preview
```

The `test` script runs `node --test` followed by `vitest run --config vitest.config.js`. `build` runs `vite build`; `preview` serves the generated `dist/` bundle. There is no configured lint script.

Environment variables

All `VITE_*` values are public client build-time configuration; they are not secret storage.

Variable	Example/default	Purpose
`VITE_DATA_MODE`	`mock`	Initial mode. Only the exact value `api` selects API mode; otherwise the app defaults to mock mode. The in-app controls can switch modes for the current session.
`VITE_API_BASE_URL`	`http://localhost:8080`	API origin; the central API module trims trailing slashes.
`VITE_API_TIMEOUT_MS`	`10000`	Request timeout in milliseconds.
`VITE_STALE_AFTER_MS`	`900000`	Measurement freshness threshold in milliseconds; must be positive.
`VITE_REFRESH_INTERVAL_MS`	`15000`	Latest-data refresh period in milliseconds; valid range is 10000-30000 inclusive.

After changing `.env`, restart the Vite development server. Never add credentials, `X-Service-Token`, or ingestion secrets to these variables.

6. API integration contract and backend expectations

The client uses the documented local backend base URL `http://localhost:8080`, configurable through `VITE_API_BASE_URL`. Requests are centralized in `src/services/api.js`; components do not construct API URLs. The client sends GET requests with an `Accept: application/json` header and does not add authentication or service-token headers.

Endpoints currently exposed by the API client

Method and path	Parameters	Expected response / current use
`GET /api/health`	None	Returns parsed JSON. The helper exists and is unit-tested; the dashboard does not call it as part of its current status indicator. The exact health response object is not validated by P3.
`GET /api/devices`	None	Client expects a JSON array. Used with the latest endpoint to load device information. The API client validates that the top-level value is an array, not the schema of each device.
`GET /api/measurements/latest?zoneId=...`	Required `zoneId`	Client expects a JSON array of measurements, not one object. Used for selected-zone dashboard data. Query value is encoded with `URLSearchParams`.

Method and path	Parameters	Expected response / current use
`GET /api/measurements`	Optional `deviceId`, `property`, `from`, `to`, `limit`	Client expects a JSON measurement array. Used by history filters. Empty optional values are omitted; non-empty values are URL encoded. A supplied limit must be an integer from 1 to 5000.

The API service rejects non-2xx responses, network failures, request/body timeouts, invalid JSON, and invalid measurement arrays with explicit errors. It supports caller cancellation. In API mode, latest device and measurement requests are made concurrently using `Promise.all`; a failure is reported rather than converted into a successful empty response.

Measurement fields and client validation

The documented measurement properties are `temperature`, `humidity`, `pressure`, and `illuminance`. The client preserves the received field names and timestamps.

Field	Client expectation/handling
`measurementId`	Required string by current response validator; used as a React key when available.
`deviceId`	Required string; shown in cards, device area, and history.
`zoneId`	Required string; used to filter/display selected-zone readings.
`gatewayId`	Optional string; displayed when present.
`property`	Required string; known properties receive labels; unknown values remain identifiable without invented meanings.
`value`	Required finite number by current response validator.
`unit`	Required string by current response validator; API-provided units are retained for measurement display.
`observedAt`	Required parseable timestamp in the current API validator; used for freshness and chart ordering.
`source`	Required to be `real` or `simulated` by the current validator. In mock mode, it is still demo data.
`quality`	Optional; current accepted values are `good`, `uncertain`, and `bad`.
`receivedAt`	Optional parseable timestamp; preserved by validation but not used to calculate freshness.

The backend documentation states that `latest` returns one record per device/property pair. The frontend handles an array. This shape and the exact field requirements still require confirmation from P2 against the actual service and future OpenAPI definition.

API contract items requiring confirmation from P2

1. Exact top-level response shapes, required/optional fields, nullable fields, source/quality values, and error response bodies.
2. Whether latest always returns an array and its behavior for an empty zone.
3. History ordering by `observedAt`, default limit 1000, and maximum 5000.
4. Whether device records expose any status/online field and what it means. P3 currently does not display a device status.
5. CORS configuration for the actual development origin (normally `http://localhost:5173`) and deployed PWA origin.
6. Whether any read-endpoint authentication is required. P3 currently assumes no read auth and sends no credentials; this must match the released API.
7. Actual endpoint availability and health response schema.
8. The eventual `contracts/openapi.yaml` and the agreed API version.

The real backend source and `contracts/openapi.yaml` were not found in the inspected repository. Therefore actual endpoint compatibility, CORS, and a successful real browser integration are **unverified**. The API client tests use mocked Fetch responses only. Never put `X-Service-Token` in the browser; it is an ingestion-side concern, not a PWA credential.

7. Mock data and demonstration behavior

`src/services/mockData.js` is the isolated fixture source. It contains two devices, four measurement properties per device, and a top-level demonstration label:

Device	Zone	Gateway	Fixture `source`
`bme280-01`	`salle-01`	`gw-01`	`real`
`sim-bme280-02`	`salle-02`	`gw-sim`	`simulated`

The example property/unit pairs are temperature/°C, humidity/%, pressure/hPa, and illuminance/lx. Values and timestamps in this file are illustrative only; they are **not actual sensor observations**. Every dashboard view in mock mode also displays a demonstration label, including records whose `source` field is `real`.

Mock history applies the selected device, property, optional date range, and limit to the same fixed fixtures. It sorts matching fixture records by `observedAt`; it does not generate a time series or invent historical measurements. The fixture's illuminance attribution to `bme280-01` is unchanged and awaits confirmation from P1/P2. The architecture reference's mention of a BH1750 does not establish that device mapping.

8. Data freshness and refresh behavior

Freshness is based on the source observation timestamp `observedAt`, not the browser request/fetch time. A successful refresh of an old measurement does not make that measurement current.

Setting/state	Actual behavior
Default stale threshold	`VITE_STALE_AFTER_MS=900000` (15 minutes). A configured value must be positive.
Default refresh interval	`VITE_REFRESH_INTERVAL_MS=15000` (15 seconds); accepted values range from 10 to 30 seconds.
Up to date	The UI label for internal freshness state `fresh`, when all displayed measurements have valid, non-future timestamps within the threshold.
Stale	At least one valid measurement is older than the threshold. Cards call this “Stale · last known reading.”
Unavailable	No measurements exist, or at least one timestamp is absent, invalid, or in the future.

The scheduler runs once immediately, then on its configured interval. If a refresh has not completed, another scheduled refresh is skipped. The dashboard effect stops the scheduler and aborts its active API request when zone/mode changes or the component unmounts. A separate five-second clock updates the displayed freshness as time passes. Errors are surfaced as API errors; they do not turn an earlier observation into a fresh one.

9. PWA implementation

Manifest and installation

`public/manifest.webmanifest` declares the application and short name “Smart Museum”, relative `./` start URL and scope, standalone display, theme color `#183c34`, background `#f4f3ee`, and SVG/192px/512px icons (including a maskable PNG reference). `src/main.jsx` registers the service worker only in the production build and only when the browser supports service workers.

`PwaControls.jsx` listens for browser online/offline and install-prompt events. The **Install app** button appears only if the browser emits `beforeinstallprompt`; the browser may instead provide its own install menu. Native installation is browser/platform dependent.

Service worker and offline behavior

`public/service-worker.js` precaches the built application entry, referenced JavaScript/CSS assets, manifest, offline page, and icons. It uses a network-first strategy for same-origin navigations, then falls back to the cached app entry and, if that is absent, `offline.html`. Same-origin scripts and styles use a cache-first lookup with a network fallback.

Requests to `/api/`, cross-origin requests, non-GET requests, and requests with an `Authorization` header are outside the worker's cache handling. Responses with `private` or `no-store` cache directives are also excluded when saving app responses. Measurement/API responses are not persisted by this worker. Already-loaded readings may remain in page memory until navigation/reload; their freshness continues to use `observedAt`.

A waiting worker announces an update. The user selects **Update app** to send `SKIP_WAITING`; the page reloads on `controllerchange`. The service worker checks for updates after page load. In development mode, the production service worker is not registered.

Verification and limitations

- Automated tests parse the manifest, check its single `id`, app identity, icon sizes, and existence/content of `offline.html`.
- A production preview and service-worker behavior have been recorded as manually checked in prior P3 QA notes.
- Physical-device installation has **not** been verified.
- The standalone `offline.html` fallback is present and unit-tested as a static file, but it was not independently forced as the displayed page in the recorded manual browser run.
- A first visit requires a successful online app-shell load before the shell can work offline.
- The app does not cache or display stored API readings as live data.

10. Testing and quality assurance

The following automated checks were run for this handoff on 2 November 2026.

Check	Result	Scope
`npm test`	**Passed: 31 Node tests and 3 React component tests**	Services/utilities/PWA assets, plus history-query labels, zone selection, and data-mode controls.
`npm run build`	**Passed**	Vite production build; 46 modules transformed.
`node --check public/service-worker.js`	**Passed**	Service-worker JavaScript syntax.
`npm audit --offline`	**Passed: 0 vulnerabilities reported**	Current dependency tree and available local audit data.
Manifest validation	**Passed through** `pwa.test.js`	Valid JSON, exactly one `id`, expected install fields and PNG dimensions.

Automated coverage includes:

- API GET endpoints, URL parameter encoding, measurement array validation, and HTTP/network/timeout/invalid-JSON errors.
- Mock device/zone filtering and history filtering/ranges/limits.
- Measurement grouping, unknown-property handling, optional metadata, and history chart point ordering.
- Recent/stale/invalid/future/missing timestamp behavior and refresh interval validation/overlap prevention.
- Manifest and offline-page assertions.
- Component behavior for history labels, zone changes, and data-mode controls.

The API tests mock Fetch; they are not end-to-end integration tests. Browser requests to the actual backend, actual CORS, and physical-device installation are unverified.

Manual QA results on record

The existing P3 QA notes report production-preview checks including a 360px viewport, active service worker, visible offline state, app-shell-only caching, and user-triggered worker update activation. The prior QA result records no page-level horizontal overflow at 360px, 768px, and 1440px. Those browser checks were **not rerun as part of generating this document**.

The install prompt is browser controlled; physical installation was not verified. There is no configured lint script.

11. UML and technical diagrams

The editable Mermaid sources are in `DOCS/diagrams/`. They describe the client's documented scope only:

Diagram	Participants/content	Source
System architecture	P1 telemetry context, P2 backend/database boundary, and P3 browser/API read path	Architecture diagram in Section 3 of this document.
Use cases	Staff and browser interactions with implemented PWA functions and documented API reads	`diagrams/pwa-use-cases.mmd`
Sequence	PWA measurement retrieval, with sensor/gateway/Node-RED ingestion shown separately as context	`diagrams/pwa-measurement-sequence.mmd`
Communication	Numbered PWA/API/MongoDB messages for dashboard and history retrieval	`diagrams/pwa-communication.mmd`

The sequence source currently declares MongoDB using Mermaid's `database` keyword. The Mermaid CLI version available during PDF generation rejects that declaration. The source remains untouched; the PDF renderer used a temporary compatible participant declaration for that page only. The editable source and generated graphic therefore have this known rendering difference.

Use-case diagram

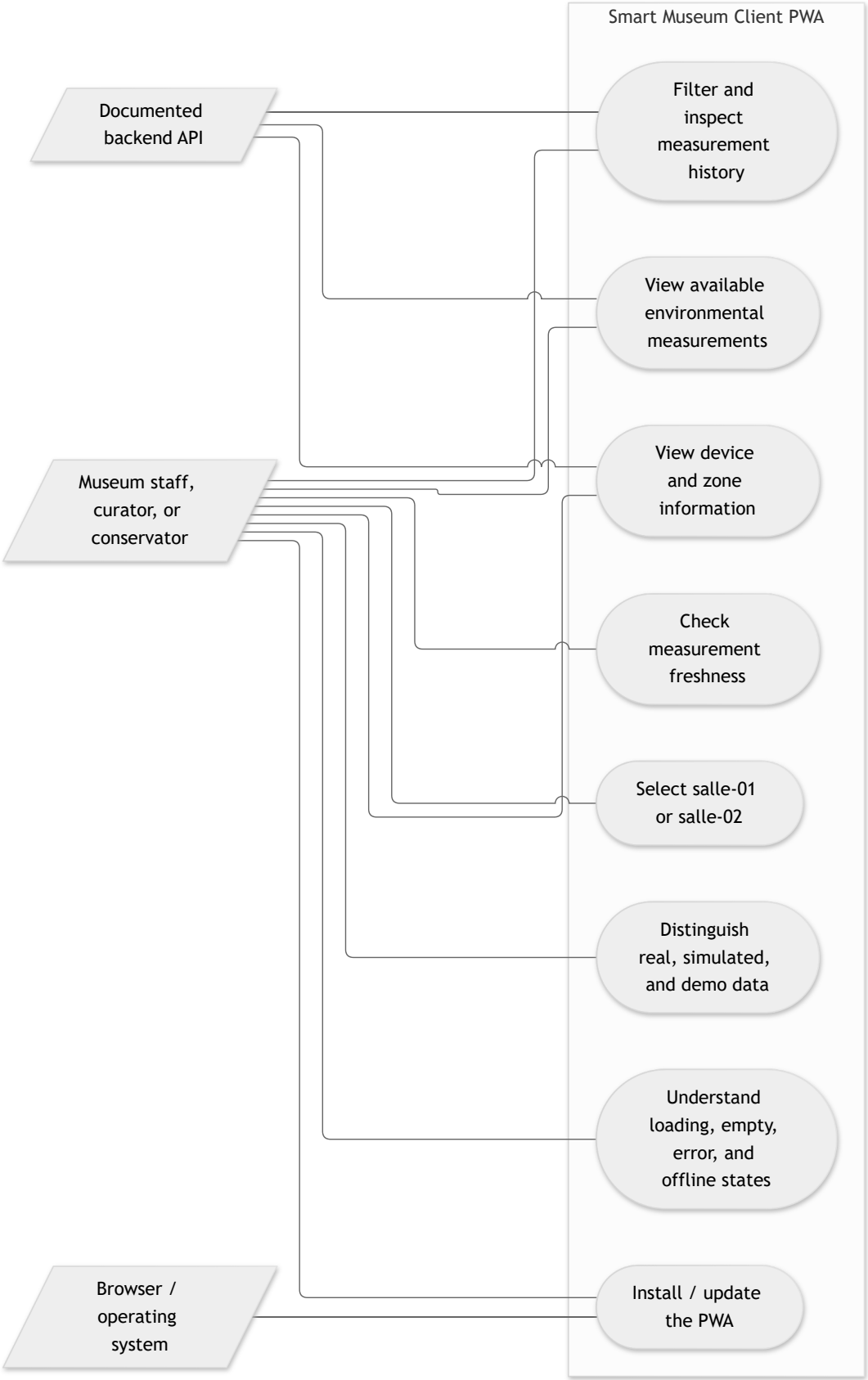


Figure 2. Implemented PWA use cases and documented API participants.

Measurement retrieval and separate ingestion sequence

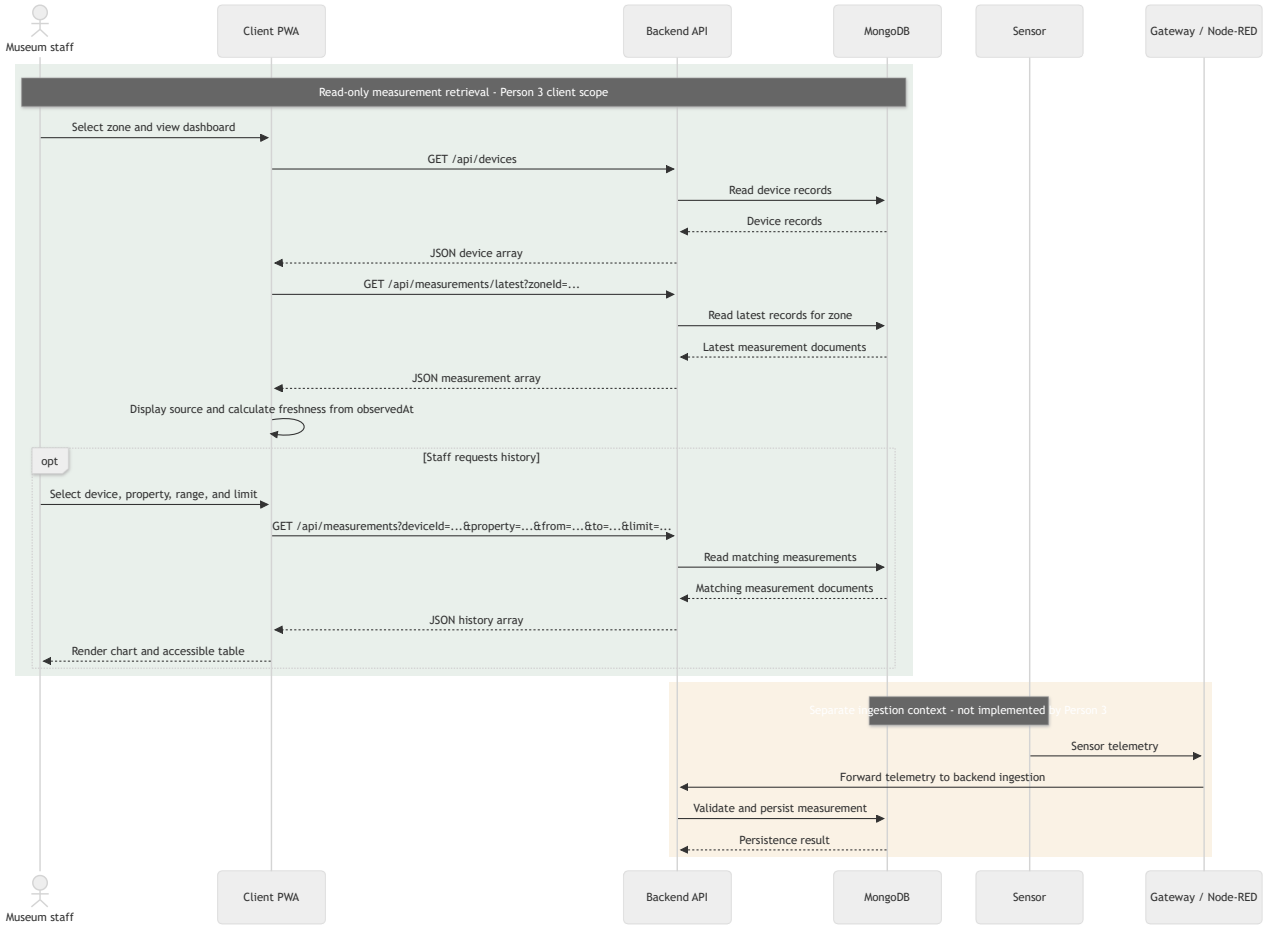


Figure 3. Read-only measurement retrieval and separate ingestion context.

Numbered communication diagram

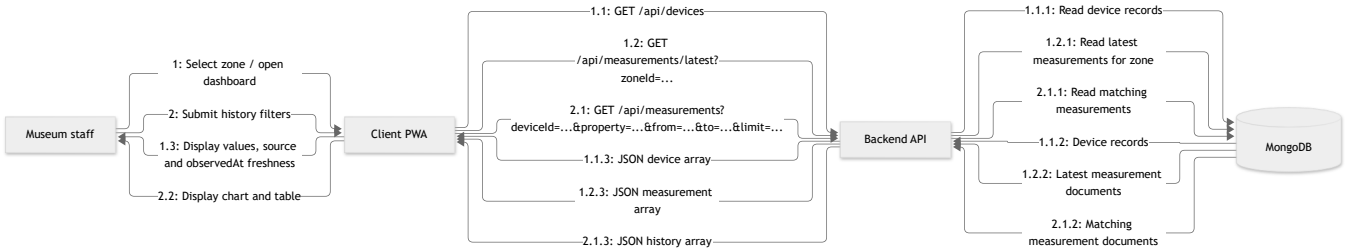


Figure 4. Numbered communication flow for dashboard and history reads.

12. Known limitations and pending decisions

Issue/decision	Owner	Why it matters	Required action	Status
Physical illuminance device attribution	P1/P2	The demo fixture associates illuminance with `bme280-01`; the architecture reference mentions BH1750 but does not confirm the record mapping.	Confirm sensor identity and device/zone attribution; then update fixtures/documentation if agreed.	Pending
Gateway/device status field	P1/P2	The PWA does not invent an online/status field.	Confirm whether status exists in device records and define its meaning.	Pending
Backend source and actual endpoints	P2	API integration currently relies on documented behavior and mocked Fetch tests.	Publish backend source and endpoint instructions; test all reads from the browser.	Pending
Shared OpenAPI contract	P2	Exact response types and optional fields cannot be verified against a contract file.	Provide and maintain `contracts/openapi.yaml`; compare it with client expectations.	Pending
Response shapes and measurement fields	P2	API client expects arrays and validates required fields/enums.	Confirm array shape, optional/null fields, source/quality values, health response, and errors.	Pending
History ordering/limits	P2	UI expects documented sorting and accepts limits up to 5000.	Confirm ascending `observedAt`, default 1000, maximum 5000, and empty-range behavior.	Pending
CORS	P2	Browser cross-origin calls may fail even when the API is reachable.	Allow and test the actual frontend origin in each environment.	Pending
Authentication	P2	Current client sends no auth headers.	Confirm whether read endpoints remain open and document any future client auth separately. Never expose ingestion token.	Pending
Real backend integration	P3, after P2	No backend is available in this repository.	Set API mode/base URL, verify browser requests/data/error behavior, and add or update integration tests where feasible.	Pending
Physical PWA installation	P3	Install prompt/platform behavior is browser-specific.	Verify installation on a supported physical device/browser.	Pending
Final API documentation	P3 after P2 confirmation	Frontend docs must reflect the agreed contract rather than assumptions.	Update handoff and P3 guide after contract review.	Pending

13. Integration procedure after P2 publishes the backend

1. Pull the agreed integration branch without overwriting teammate work.
2. Inspect P2's backend source, startup guide, and actual `contracts/openapi.yaml`.
3. Compare endpoint paths, response arrays, fields, optional/null values, limits, sorting, health response, and error bodies with `src/services/api.js`.

4. Start MongoDB and the backend using P2's instructions; do not assume commands or deployment configuration that P2 has not supplied.
5. Check backend health and each documented read endpoint with P2-approved requests. Verify actual latest and history response arrays.
6. Set `.env` to the real `VITE_API_BASE_URL` and `VITE_DATA_MODE=api`. Keep all credentials and ingestion tokens out of frontend variables.
7. Restart Vite and record the actual origin printed in the terminal.
8. Configure P2's CORS allowlist for that exact origin and the intended deployed PWA origin.
9. Open browser Network and Console panels. Check health/devices/latest/history requests, status codes, JSON parsing, and CORS preflight/error behavior.
10. Test both zones, optional metadata, empty data, invalid fields, API errors, history date filters/limits, offline state, and stale timestamps.
11. Run `npm test` and `npm run build`; add tests only for confirmed behavior and keep API contract changes documented.
12. Update `DOCS/pwa-client.md` and this handoff with confirmed contract, CORS, and verification results.

14. Handoff summary and responsibilities

Delivered by P3

- Responsive React/Vite dashboard with zone selection, measurement/device presentation, provenance labels, freshness status, and history view.
- Mock fixtures and a centralized, configurable API client for documented read-only endpoints.
- PWA manifest, icons, app-shell service worker, offline experience, and browser-controlled install/update UI.
- Automated service/utility/component tests and P3 guide/diagrams.

Ready for review

- Frontend mock mode and its tests/build.
- API client shape and error handling as implemented against mocked responses.
- P3 architecture, UI behavior, freshness defaults, and app-shell strategy.

Not verified or pending

- Live backend behavior, actual endpoint responses, CORS, and API contract.
- Device online/status fields and illuminance physical sensor attribution.
- Physical-device installation and standalone fallback display behavior.
- Final alignment with `contracts/openapi.yaml` once P2 supplies it.

Responsibility handoff

- **P1:** confirm illuminance sensor/device mapping and device/gateway status semantics.
- **P2:** publish backend and OpenAPI contract; confirm endpoints, response shapes, limits, CORS, and authentication expectations.
- **P3:** connect and browser-test against the agreed backend, confirm installation on a supported device, and update documentation.

Team handoff message: “P3 has delivered a mock-capable React/Vite client PWA, its documented read-only API integration boundary, automated tests, and editable diagrams. Mock mode/build are verified; real backend/CORS are not. Please confirm the physical illuminance mapping with P1 and publish the backend/OpenAPI contract with P2 before live integration.”